ME-502: Applied Numerical Methods

Assignment 1

Submitted by-

Apoorva Singh

2025MEM1003

An Introduction to Numerical analysis using Python-

This report demonstrates the use of **Python** for exploring numerical differentiation methods. Here are the step-by-step instructions for the successful execution of the program to achieve the desired result.

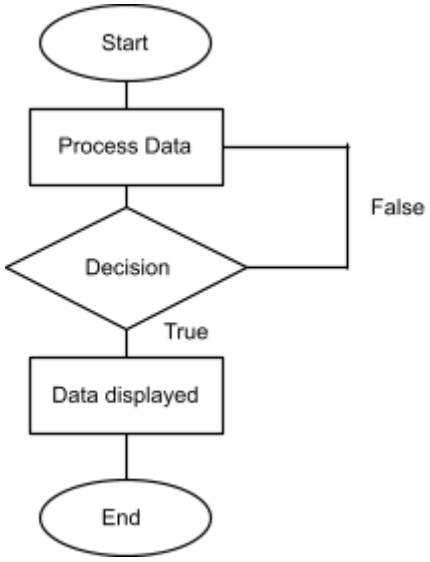
A. **Installing Python Modules**- Python Modules contain a set of functions, classes and variables that increase reusability and reduce the complexity of the code. Before being used in the code, they must be installed via Command Prompt as shown in Fig 1.1; further, here is the list of modules to be installed-

1. **Matplotlib**- Matplotlib graphs your data on Figures (e.g., windows, Jupyter widgets, etc.), each of which can contain one or more axes, an area where points can be specified in terms of x-y coordinates as specified by the user.
2. **Sympy**- SymPy is a Python library for symbolic mathematical functions as a computer algebra system (CAS), allowing users to perform various mathematical operations symbolically rather than numerically.
3. **Numpy**- NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines.

B. **Writing the Python Program**- Generally, to implement numerical differentiation, we first write a Python program that:

1. Defines the mathematical function $f(x)$.
2. Plots the required graph/figures.

Here is the general flowchart of the program, it may vary from problem to problem.



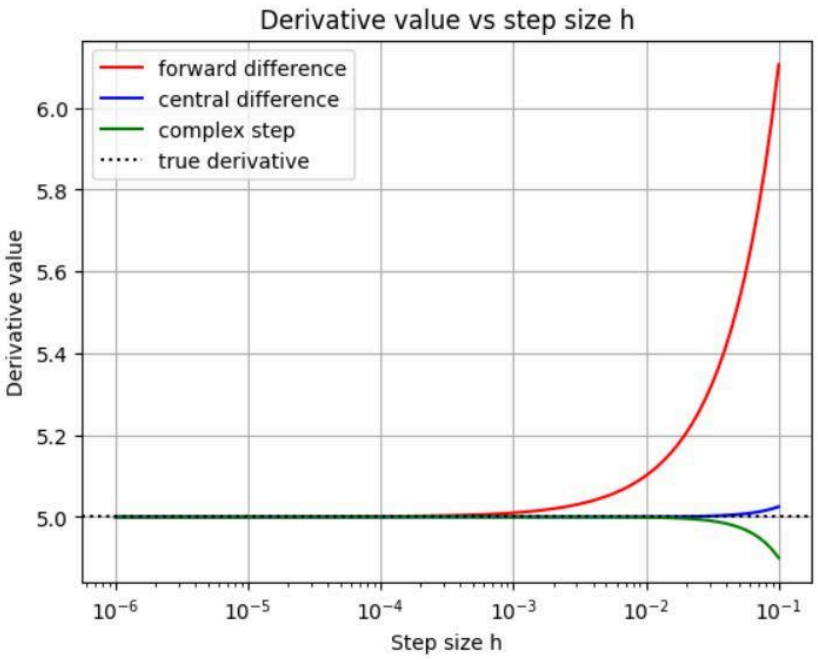
Q1.

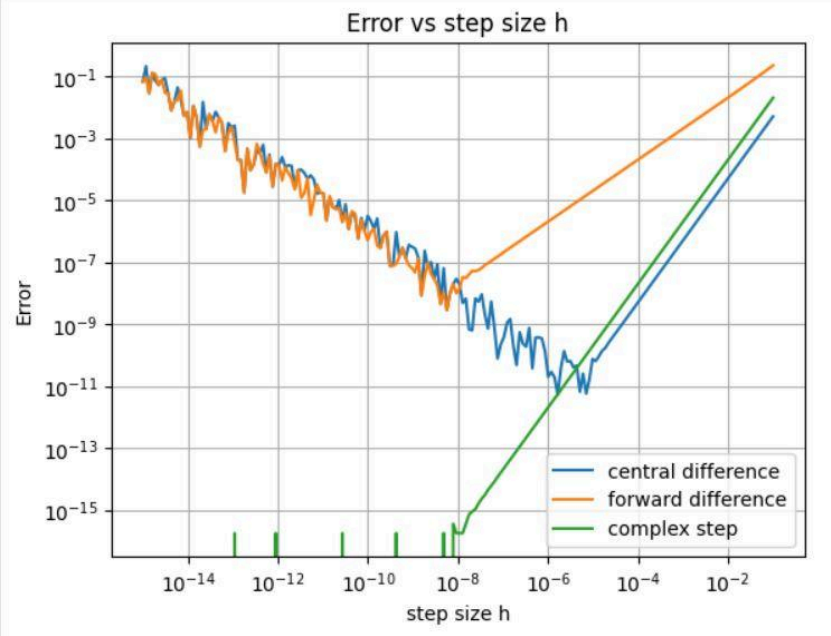
To compute the derivatives of following functions using Forward, Central difference and complex number method.

Solution -

1. For $x^a + 12.5$ where $a = 5$

Output-





a. Input parameters-

$f(x) = x^a + 12.5$

$x = 1$

$a = 5$

Step size, $h = 10^{-6}\ to\ 10^{-1}$ into 100 divisions

b. Output parameters-

Actual value of derivative function = 5

c. Observation-

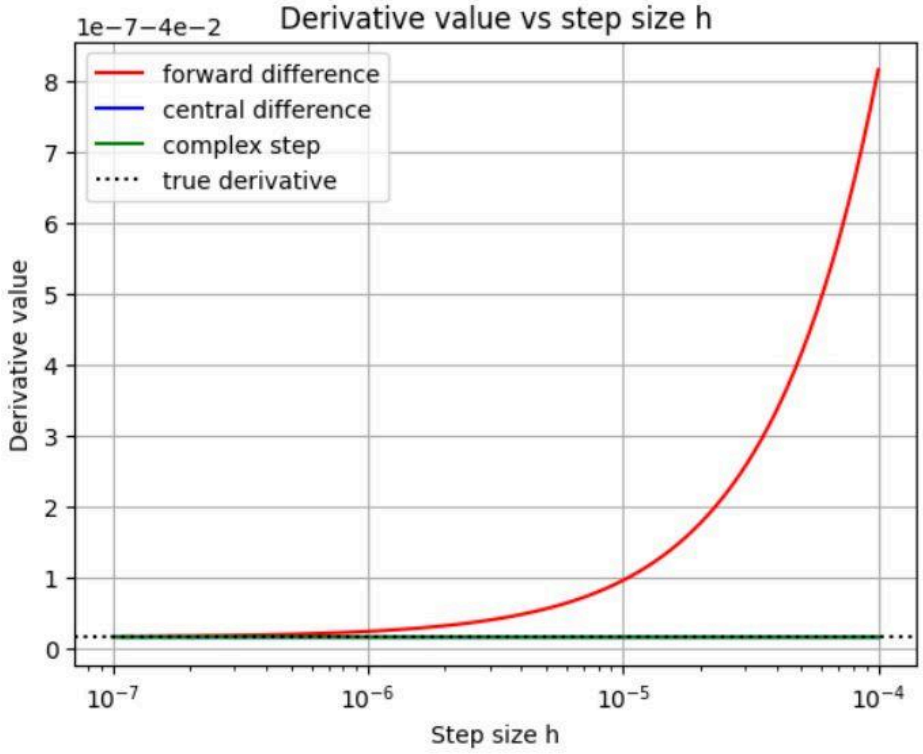
1. As can be observed, the values converge at smaller values of "h" (nearly at 10^{-3}). Further for larger values of "h", especially the Forward difference method followed by Central difference and least of all complex step method which tracks the true derivative very accurately.
2. Observing the "error vs step size h", we can see the effects of the size of "h" being that for extremely small values, the rounding error dominates for Central (on the left side of 10^{-5}) and Forward difference (on the left side of 10^{-8}) while the Complex method (at 10^{-8}) remains mostly intact though it shows fluctuations. The fluctuations are observed due to the 16^{th} decimal digit limit in python and oscillations that come from it.
3. For larger values of "h", truncation error dominates with Forward difference showing the largest deviation.

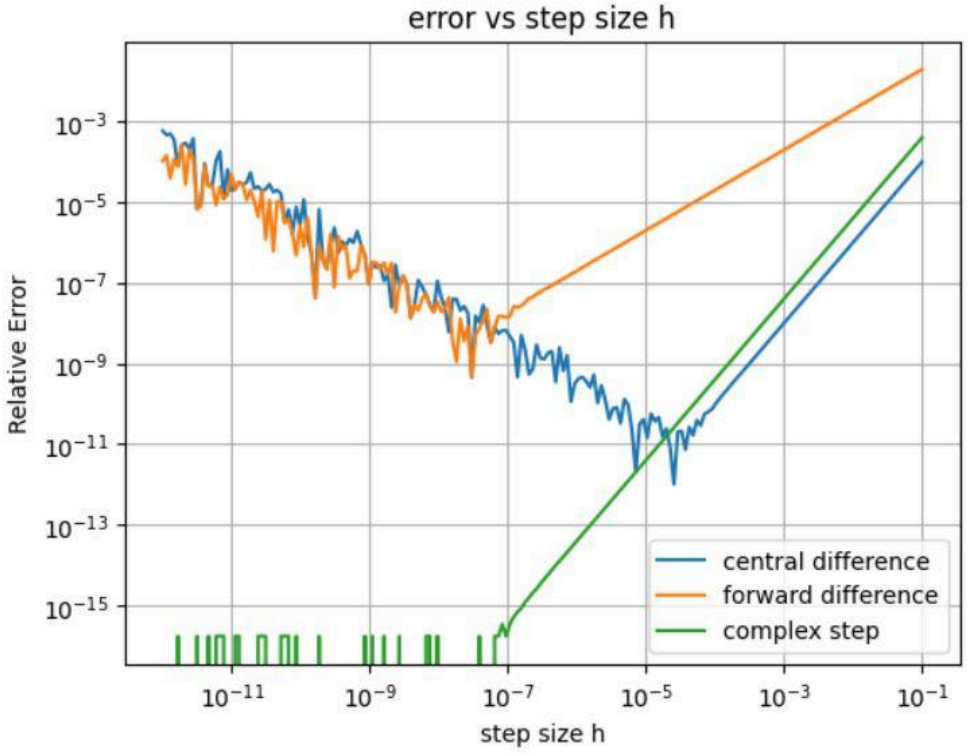
d. Discussion-

1. For "derivative vs step size h", the actual derivative value is depicted by the dashed black line, acting as a reference value.
2. Three numerical analysis methods have been used to calculate the approximate derivation- Forward difference method, Central difference method and complex step method.
3. Due to extremely small value of "h", x-axis is taken in $log(10)$ for ease of plotting.

2. For $1/x + b$ where *b=1e-6*

Output-





a. Input parameters-

$f(x) = 1/(x + b)$

$x = 5$

a = 1e-6

Step size, $h = 10^{-12}$ to 10^{-1} into 100 divisions

b. Output parameters-

Actual value of derivative function = 0.582

c. Observation-

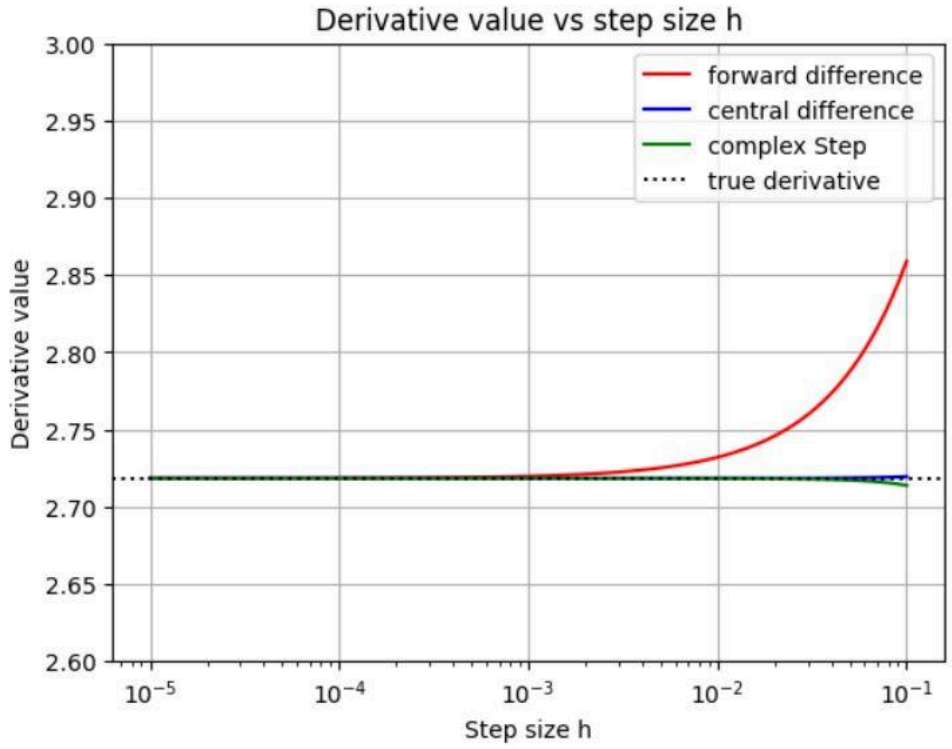
1. As can be observed, the values converge at smaller values of "h" (nearly at 10^{-6}). Further for larger values of "h", especially the Forward difference method followed by Central difference and least of all complex step method which tracks the true derivative very accurately.
2. Observing the "error vs step size h", we can see the effects of the size of "h" being that for extremely small values, the rounding error dominates for Central (on the left side of 10^{-4}) and Forward difference (on the left side of 10^{-7}) while the Complex method (at 10^{-7}) remains mostly intact though it shows fluctuations. The fluctuations are observed due to the 16^{th} decimal digit limit in python and oscillations that come from it.
3. For larger values of "h", truncation error dominates with Forward difference showing the largest deviation.

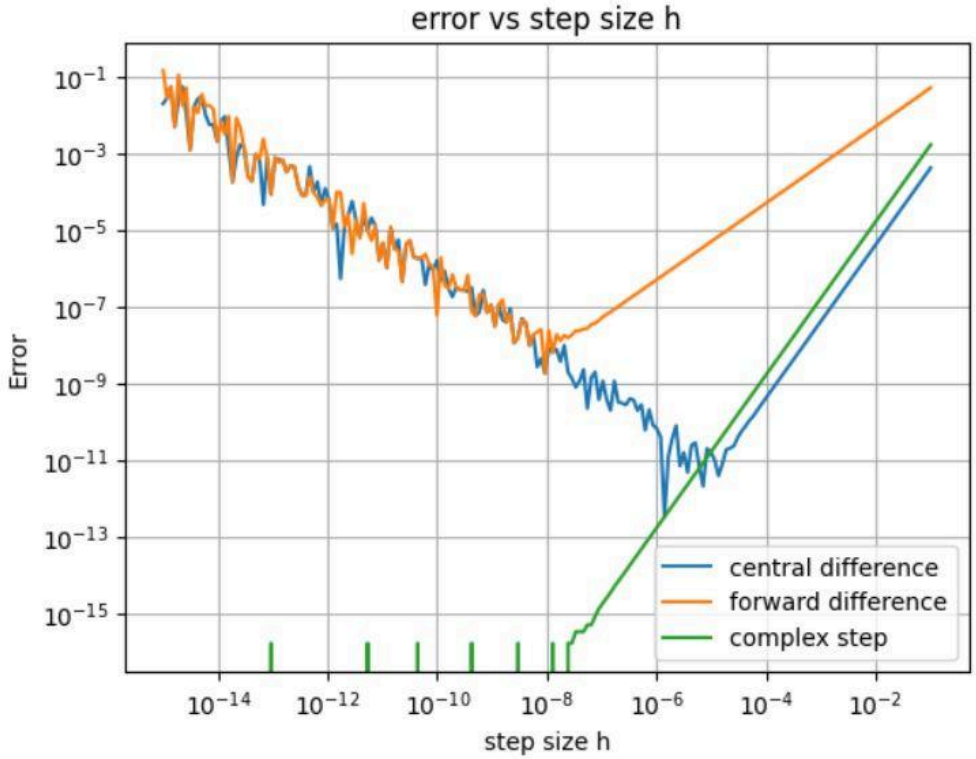
d. Discussion-

1. For "derivative vs step size h", the actual derivative value is depicted by the dashed black line, acting as a reference value.
2. Three numerical analysis methods have been used to calculate the approximate derivation- Forward difference method, Central difference method and complex step method.
3. Due to extremely small value of "h", x-axis is taken in $log(10)$ for ease of plotting.

3. For e^x where $x=1$

Output-





a. Input parameters-

$f(x) = e^x$

$x = 1$

Step size, $h = 10^{-15}\ to\ 10^{-1}$ into 200 divisions

b. Output parameters-

Actual value of derivative function = 2.718

c. Observation-

1. As can be observed, the values converge at smaller values of "h" (nearly at 10^{-3}). Further for larger values of "h", especially the Forward difference method followed by Central difference and least of all complex step method which tracks the true derivative very accurately.
2. Observing the "error vs step size h", we can see the effects of the size of "h" being that for extremely small values, the rounding error dominates for Central (on the left side of 10^{-5}) and Forward difference (on the left side of 10^{-8}) while the Complex method (at 10^{-8}) remains mostly intact though it shows fluctuations. The fluctuations are observed due to the 16^{th} decimal digit limit in python and oscillations that come from it.
3. For larger values of "h", truncation error dominates with Forward difference showing the largest deviation.

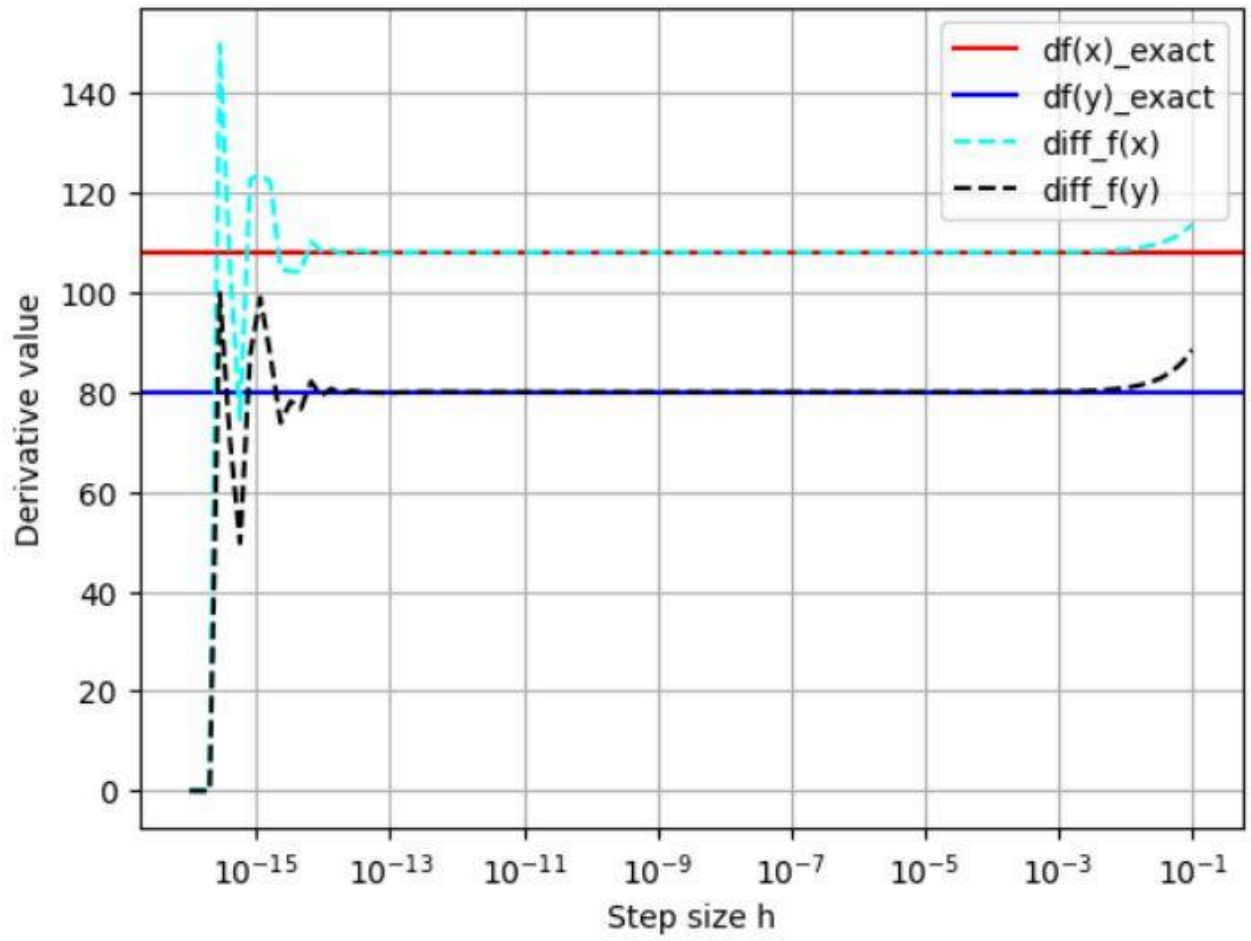
d. Discussion-

1. For "derivative vs step size h", the actual derivative value is depicted by the dashed black line, acting as a reference value.
2. Three numerical analysis methods have been used to calculate the approximate derivation- Forward difference method, Central difference method and complex step method.
3. Due to extremely small value of "h", x-axis is taken in $log(10)$ for ease of plotting.

Q2.

Now, we are going to extend the above program to solve for the two variable cases. Here, you can choose any one method either forward difference or backward difference. To validate, calculate the value of the derivative and compare with the exact value. For this purpose, you can take the function $x^a + y^b$*, here,* $a \in [4, 10]$ *and* $b \in [4, 10]$*. Take any value of x, y, a, b.*

Solution-



a. Input parameters-

f(x,y) = $x^a + y^b$

a=4

b=5

For x=3 and y=2

Step size, *h* = 10^{-16} *to* 10^{-1} into 100 divisions

b. Output parameters-

Actual value of derivative function at

1. $f'(x)$=108
2. $f'(y)$=80
3. $d^2f/dx.dy$=0

c. Observation-

1. For step size "h", convergence happens between (10^{-13} to 10^{-3}).
2. For smaller values of "h", rounding error fluctuates between extreme values.
3. For larger values of "h", truncation error dominates with Forward difference showing the largest deviation.

d. Discussion-

1. The Forward difference method is used for solving this equation
2. Due to extremely small value of "h", x-axis is taken in $log(10)$ for ease of plotting.

Q3.

In this program, we are going to write a generic program to find the roots of an equation using the bisection method. For the verification purpose, you can use the following functions.

Solution-

1.

a. Input parameters-

f(x)=$(x-4)^2-1$

a=4

b=10

Tolerance = 0.001

Iterations = 100

b. Output parameters-

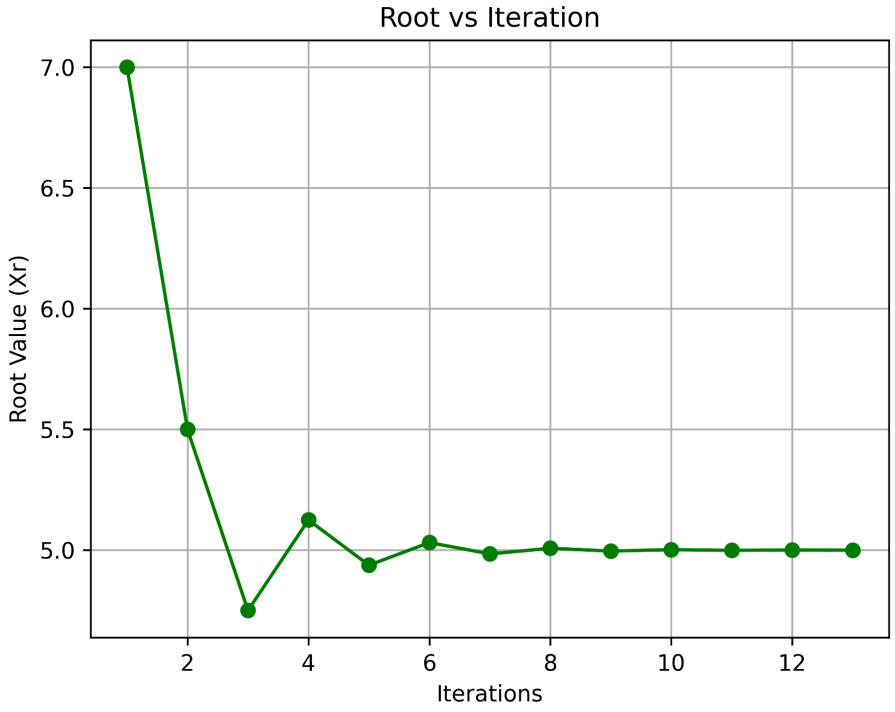
The root is 7.0 at 1 iterations
The root is 5.5 at 2 iterations
The root is 4.75 at 3 iterations
The root is 5.125 at 4 iterations
The root is 4.9375 at 5 iterations
The root is 5.03125 at 6 iterations
The root is 4.984375 at 7 iterations
The root is 5.0078125 at 8 iterations
The root is 4.99609375 at 9 iterations
The root is 5.001953125 at 10 iterations
The root is 4.9990234375 at 11 iterations
The root is 5.00048828125 at 12 iterations
The root is 4.999755859375 at 13 iterations
Actual root: 5

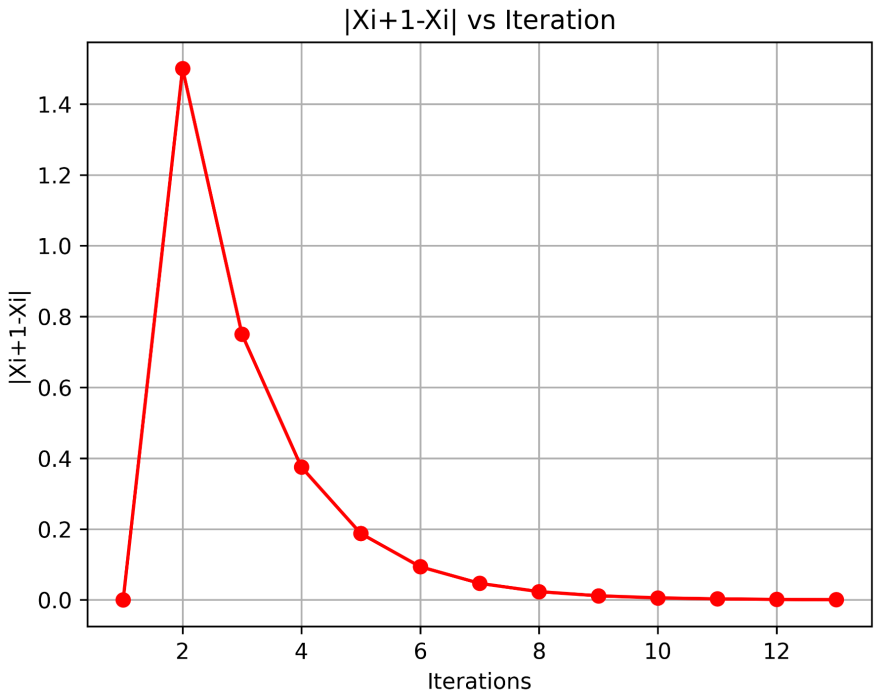
c. Observation-

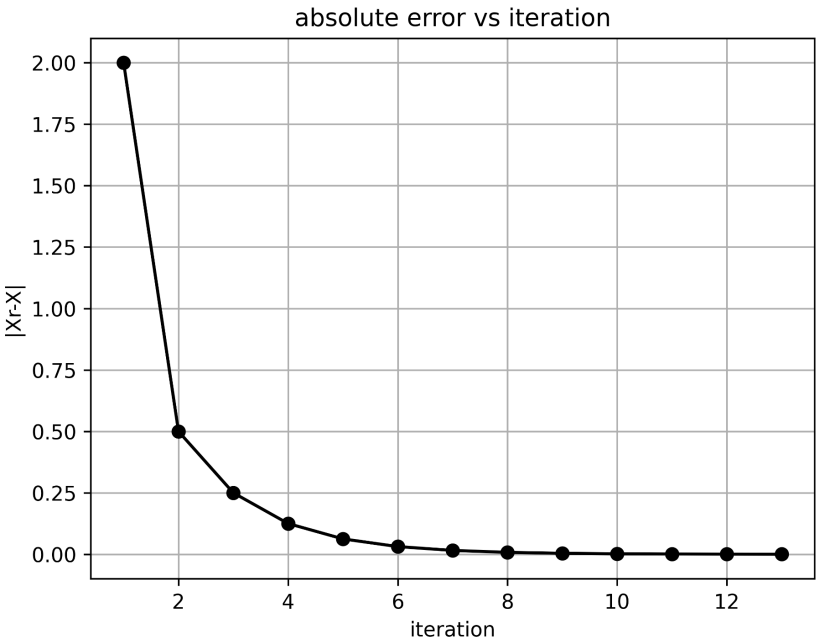
1. After 13 iterations, the value of the approximated root is within the tolerance of the actual root.
2. The difference between successive roots keeps decreasing until it reaches the scale of machine epsilon, at which point the Python program can no longer distinguish the values.

d. Discussion-

1. To prevent inexhaustive calculations (since the actual root may never be reached due to rounding error or machine epsilon) hence why a limit to iterations or tolerance value is set up.


Root vs Iteration
Root Value (Xr)
7.0
6.5
6.0
5.5
5.0
2
4
6
8
10
12
Iterations


|Xi+1-Xi| vs Iteration
|Xi+1-Xi|
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
2
4
6
8
10
12
Iterations


absolute error vs iteration
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
|Xr-X|
2
4
6
8
10
12
iteration

2.

a. Input parameters-

f(x)=$\sin x \cos x + b$

a=1

b=10

Tolerance = 0.001

Iterations = 100

b. Output parameters-

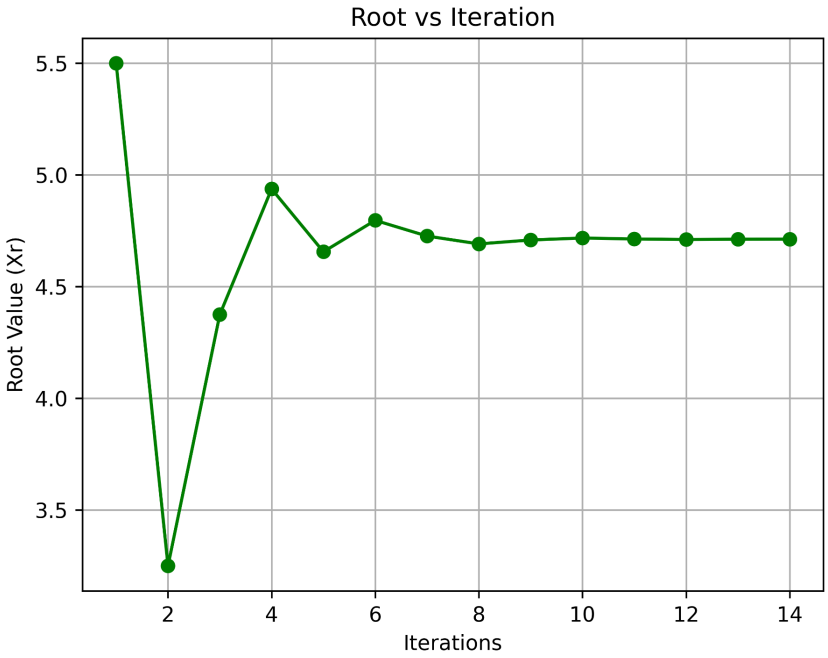
the root is 5.5 at 1 iteration.
the root is 3.25 at 2 iteration.
the root is 4.375 at 3 iteration.
the root is 4.9375 at 4 iteration.
the root is 4.65625 at 5 iteration.
the root is 4.796875 at 6 iteration.
the root is 4.7265625 at 7 iteration.
the root is 4.69140625 at 8 iteration.
the root is 4.708984375 at 9 iteration.
the root is 4.7177734375 at 10 iteration.
the root is 4.71337890625 at 11 iteration.
the root is 4.711181640625 at 12 iteration.
the root is 4.7122802734375 at 13 iteration.
the root is 4.71282958984375 at 14 iteration.
Actual root : 4.71238898038468

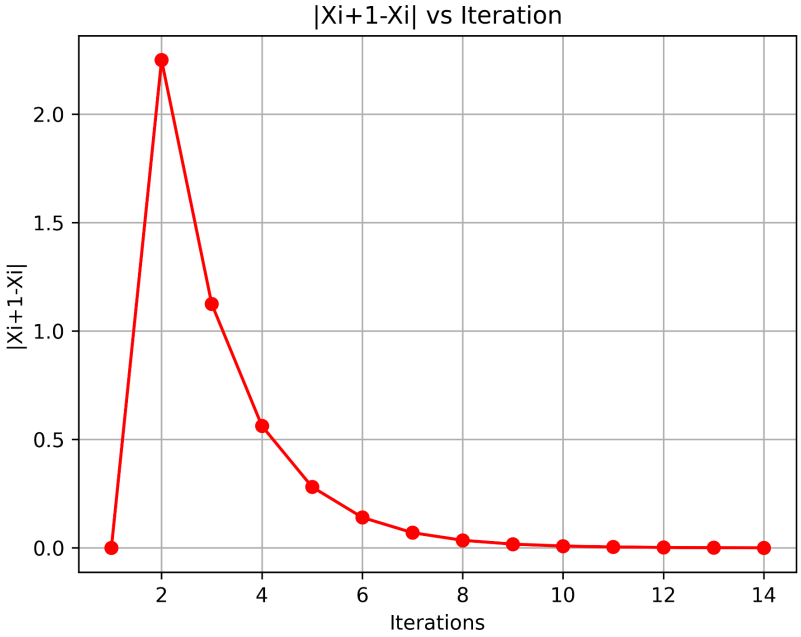
c. Observation-

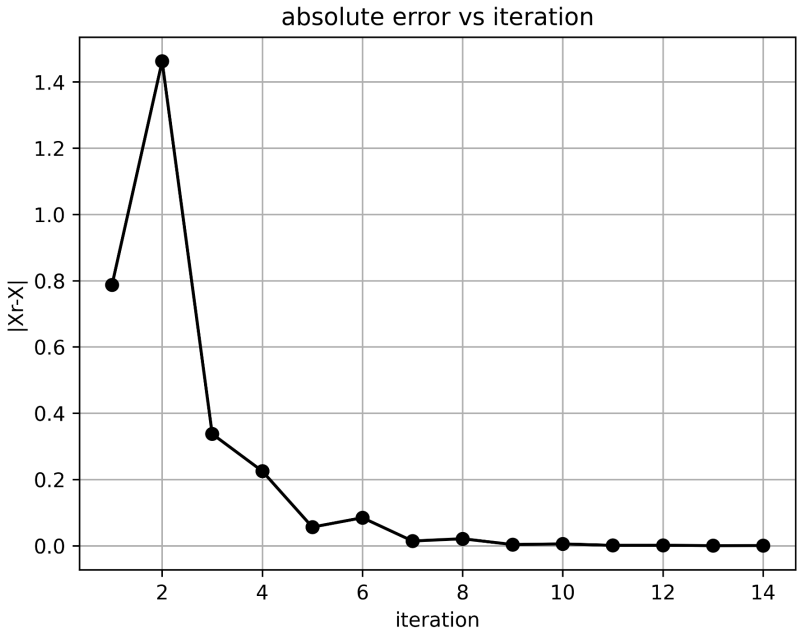
1. After 14 iterations, the value of the approximated root is within the tolerance of the actual root = 4.71238898038468
2. The difference between successive roots keeps decreasing until it reaches the scale of machine epsilon, at which point the Python program can no longer distinguish the values.

d. Discussion-

1. To prevent inexhaustive calculations (since the actual root may never be reached due to rounding error or machine epsilon) hence why a limit to iterations or tolerance value is set up.


Root vs Iteration
Root Value (Xr)
5.5
5.0
4.5
4.0
3.5
2
4
6
8
10
12
14
Iterations


|Xi+1-Xi| vs Iteration
|Xi+1-Xi|
2.0
1.5
1.0
0.5
0.0
2
4
6
8
10
12
14
Iterations


absolute error vs iteration
|Xr-X|
iteration
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
2
4
6
8
10
12
14

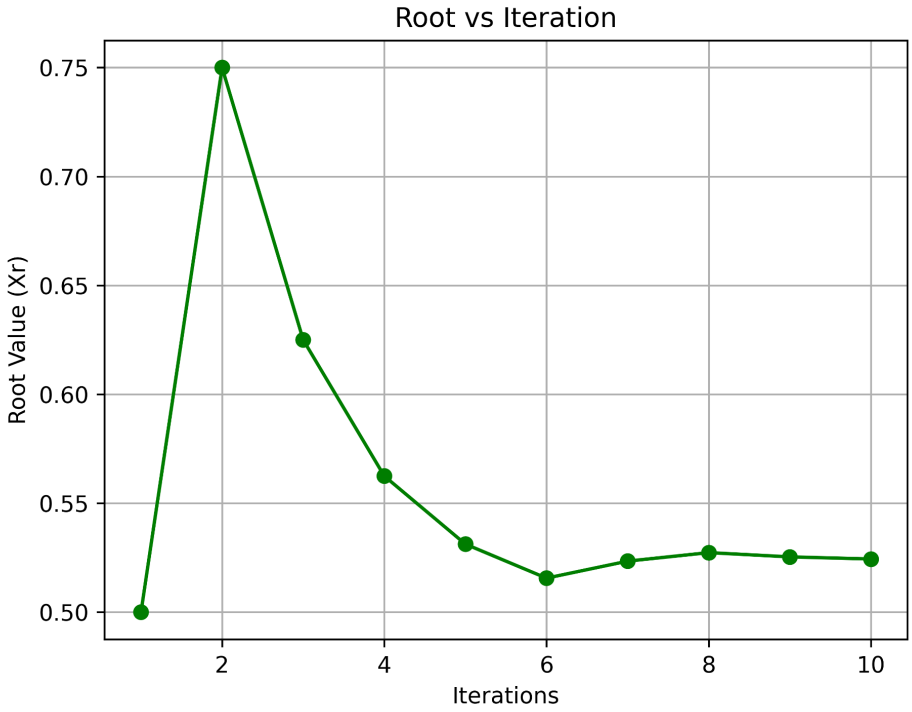
3.

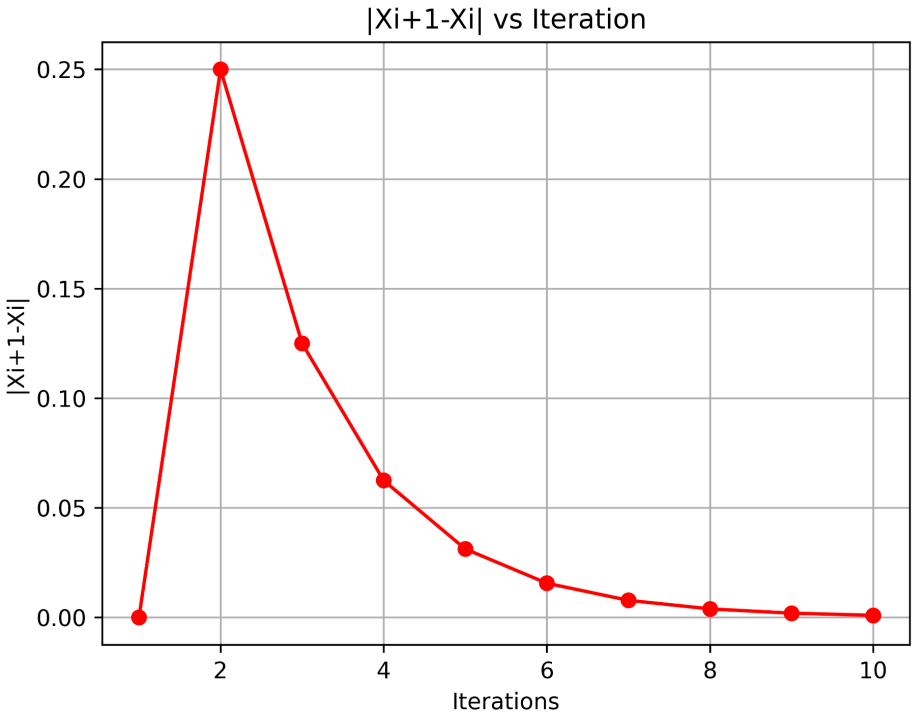
a. Input parameters-
   f(x)=$sin\ x\ -\ 0.5$
   a=0
   b=1
   Tolerance = 0.001
   Iterations = 100

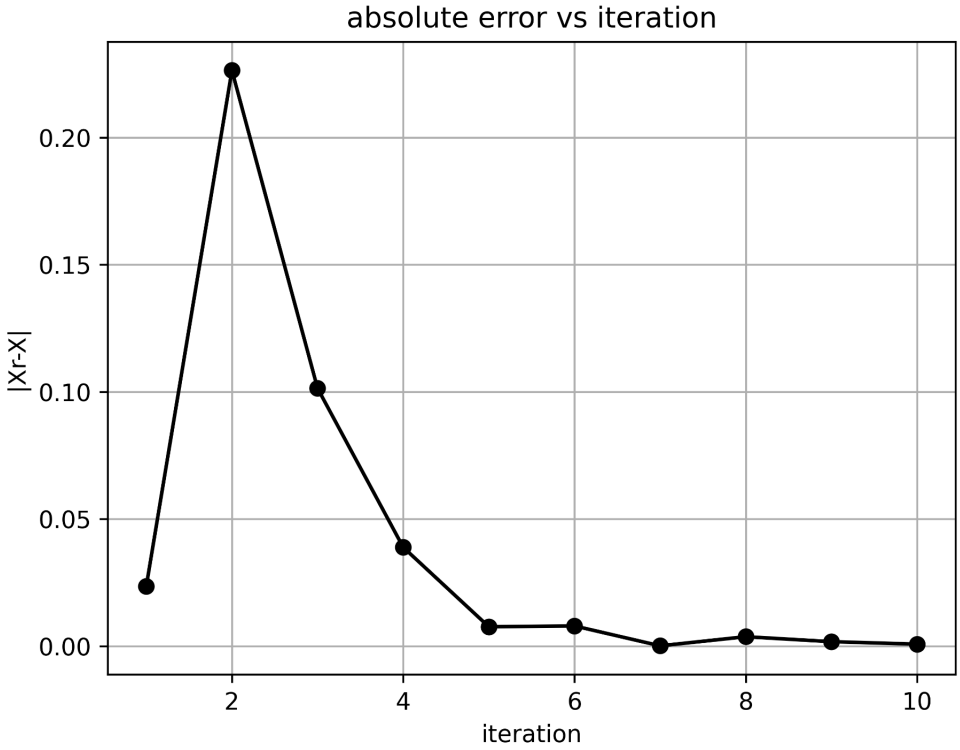
b. Output parameters-
   The root is 0.5 at 1 iterations
   The root is 0.75 at 2 iterations
   The root is 0.625 at 3 iterations
   The root is 0.5625 at 4 iterations
   The root is 0.53125 at 5 iterations
   The root is 0.515625 at 6 iterations
   The root is 0.5234375 at 7 iterations
   The root is 0.52734375 at 8 iterations
   The root is 0.525390625 at 9 iterations
   The root is 0.5244140625 at 10 iterations
   Actual root: 0.5236

c. Observation-
   1. After 10 iterations, the value of the approximated root is within the tolerance of the actual root = 0.5236
   2. The difference between successive roots keeps decreasing until it reaches the scale of machine epsilon, at which point the Python program can no longer distinguish the values.

d. Discussion-
   1. To prevent inexhaustive calculations (since the actual root may never be reached due to rounding error or machine epsilon) hence why a limit to iterations or tolerance value is set up.


Root vs Iteration
Root Value (Xr)
0.75
0.70
0.65
0.60
0.55
0.50
2
4
6
8
10
Iterations


|Xi+1-Xi| vs Iteration
|Xi+1-Xi|
0.25
0.20
0.15
0.10
0.05
0.00
2
4
6
8
10
Iterations


absolute error vs iteration
|Xr-X|
0.20
0.15
0.10
0.05
0.00
2
4
6
8
10
iteration

Q4.

In this program, we are going to write a generic program to find the roots of an equation using the bisection method. For the verification purpose, you can use the following functions.

Solution-

1.

1. Input parameters-

   f(x)=$(x - 4)^2 - 1$
   df(x)=$2 * (x - 4)$
   Tolerance = 0.001
   Iterations = 100

2. Output parameters-

   The root is 5.25 at 1 iterations.

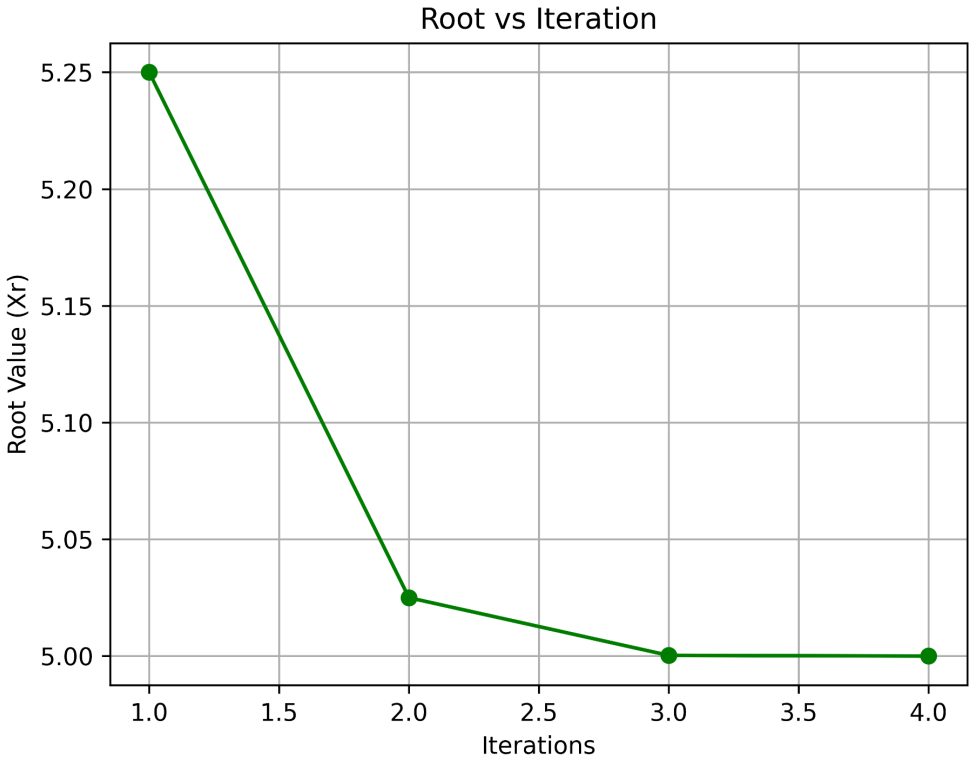
   The root is 5.025 at 2 iterations.

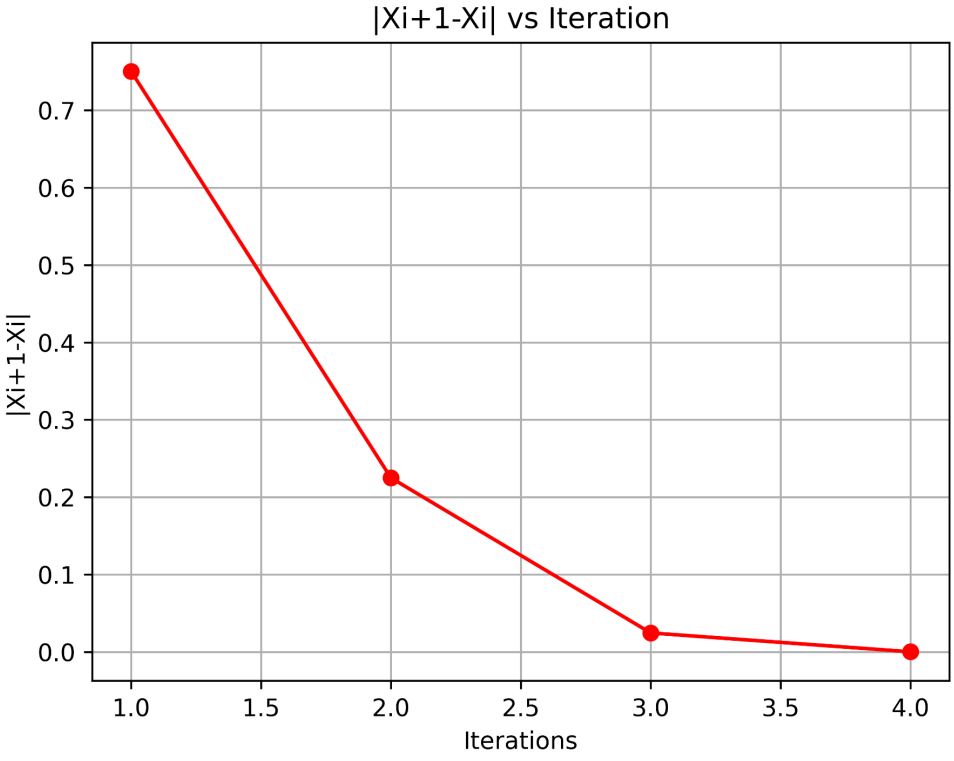
   The root is 5.00030487804878 at 3 iterations.

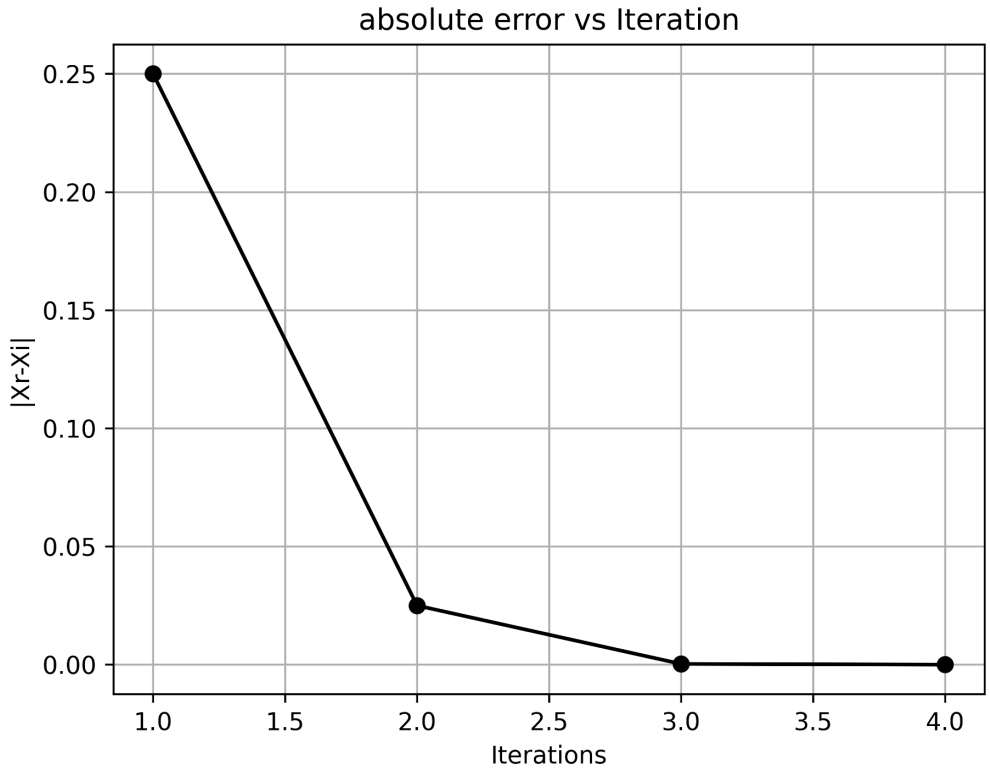
   with actual root as 5

3. Observation-
   1. After 10 iterations, the value of the approximated root is within the tolerance of the actual root = 5
   2. The difference between successive roots keeps decreasing until it reaches the scale of machine epsilon, at which point the Python program can no longer distinguish the values.

4. Discussion-
   1. To prevent inexhaustive calculations (since the actual root may never be reached due to rounding error or machine epsilon) hence why a limit to iterations or tolerance value is set up.
   2. As compared to Bi-section method, we are able to get the solution within a few steps.


Root vs Iteration
Root Value (Xr)
5.25
5.20
5.15
5.10
5.05
5.00
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Iterations


|Xi+1-Xi| vs Iteration
|Xi+1-Xi|
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Iterations


absolute error vs Iteration
0.25
0.20
0.15
0.10
0.05
0.00
|Xr-Xi|
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Iterations

2.

1. Input parameters-

   $f(x) = \sin x \cos x + b$

   $df(x) = \cos(2x)$

   Tolerance = 0.001

   Iterations = 100

2. Output parameters-

   The root is 8.88141952788763 at 1 iterations.

   The root is 9.832338692578917 at 2 iterations.

   The root is 9.30169555242616 at 3 iterations.

   The root is 9.427325883717728 at 4 iterations.

   The root is 9.42477793871463 at 5 iterations.

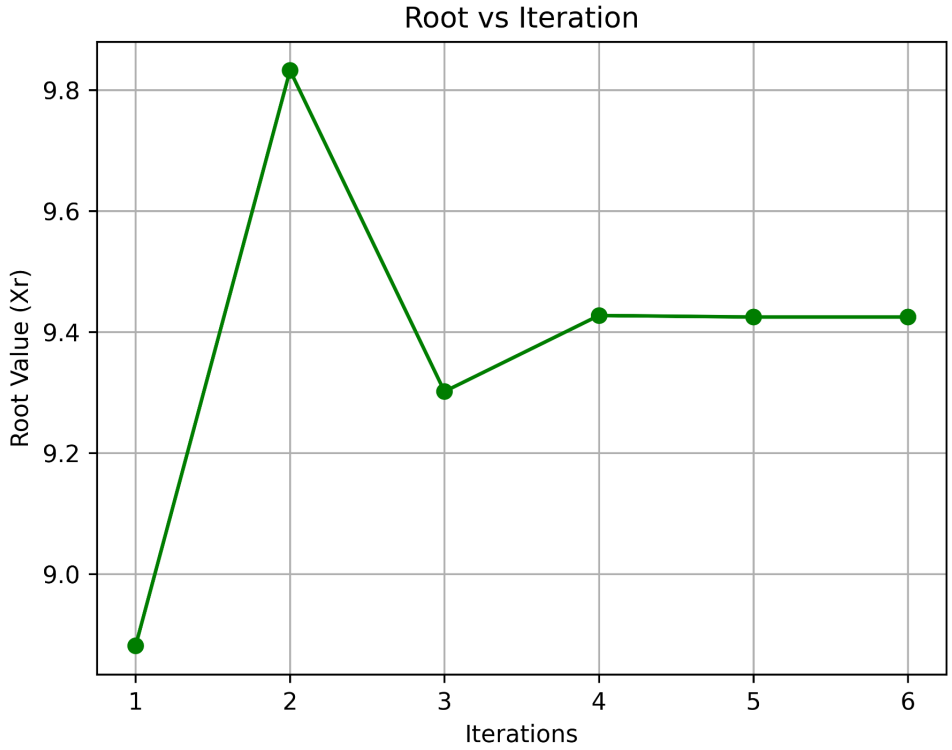
   with actual root as 9.4247779607693

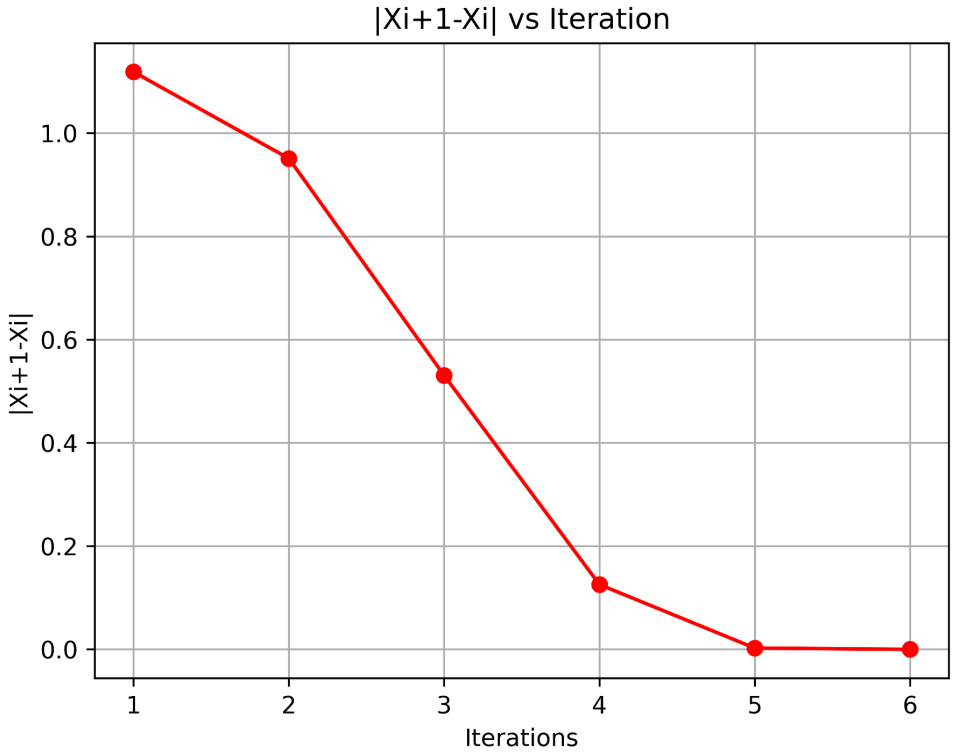
3. Observation-

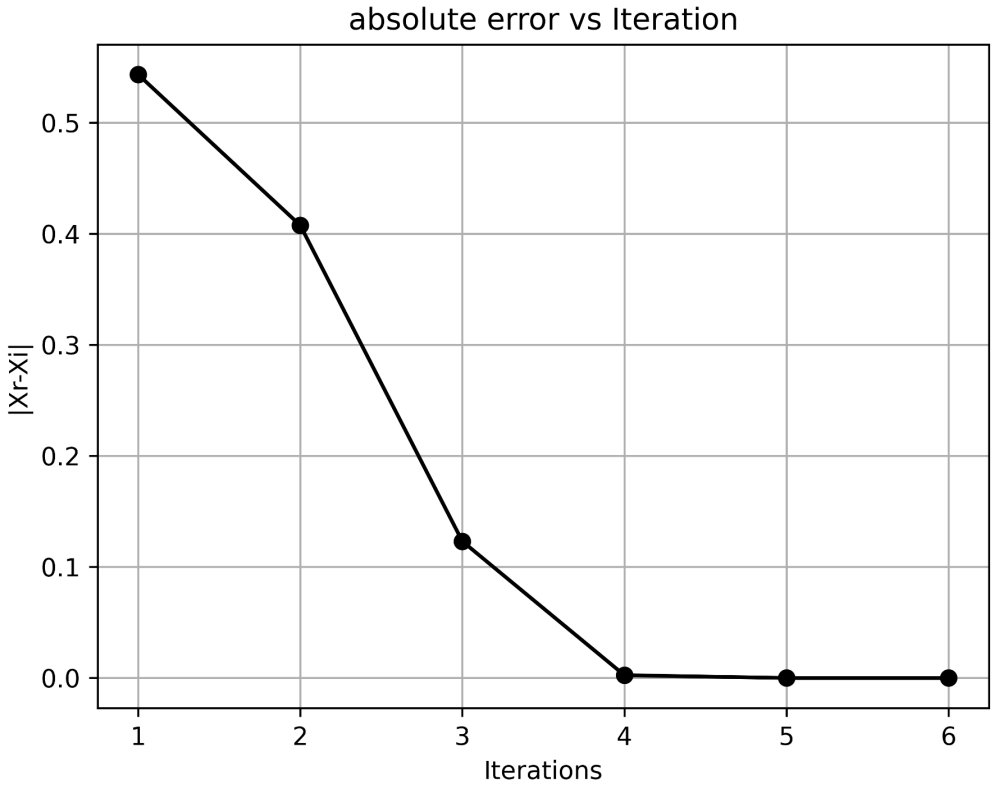
   1. After 5 iterations, the value of the approximated root is within the tolerance of the actual root = 9.4247779607693
   2. The difference between successive roots keeps decreasing until it reaches the scale of machine epsilon, at which point the Python program can no longer distinguish the values.

4. Discussion-

   1. To prevent inexhaustive calculations (since the actual root may never be reached due to rounding error or machine epsilon) hence why a limit to iterations or tolerance value is set up.


Root vs Iteration
Root Value (Xr)
9.8
9.6
9.4
9.2
9.0
1
2
3
4
5
6
Iterations


|Xi+1-Xi| vs Iteration
|Xi+1-Xi|
1.0
0.8
0.6
0.4
0.2
0.0
1
2
3
4
5
6
Iterations


absolute error vs Iteration
|Xr-Xi|
0.0
0.1
0.2
0.3
0.4
0.5
1
2
3
4
5
6
Iterations

3.

1. Input parameters-

   f(x)=$sin\ x\ -\ 0.5$

   df(x)=$cos(x)$

   Tolerance = 0.001

   Iterations = 100

2. Output parameters-

   The root is 0.36800013418556066 at 1 iterations.

   The root is 0.5183136441498523 at 2 iterations.

   The root is 0.5235907856809277 at 3 iterations.

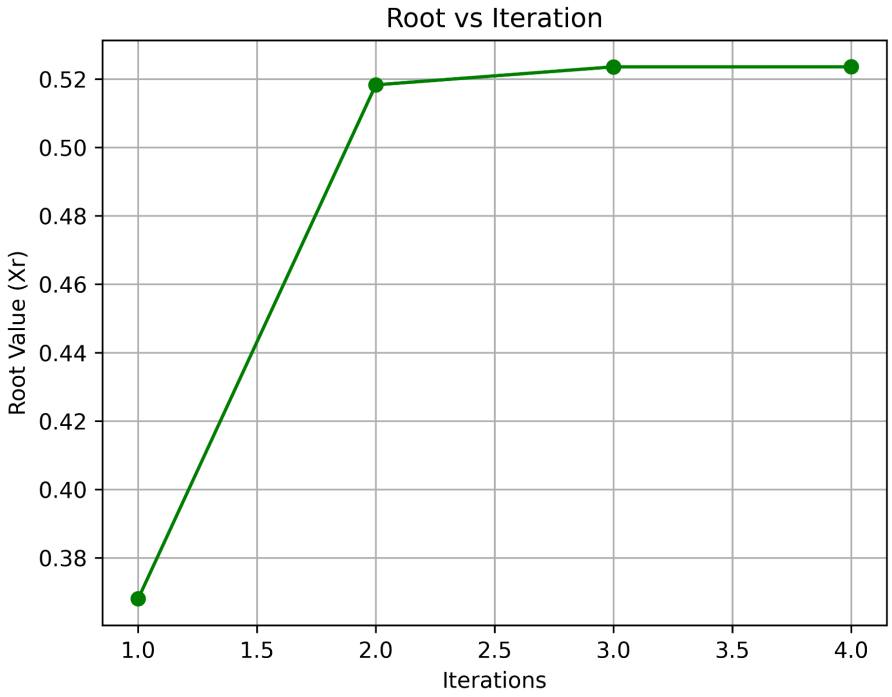
   with actual root as 0.5236

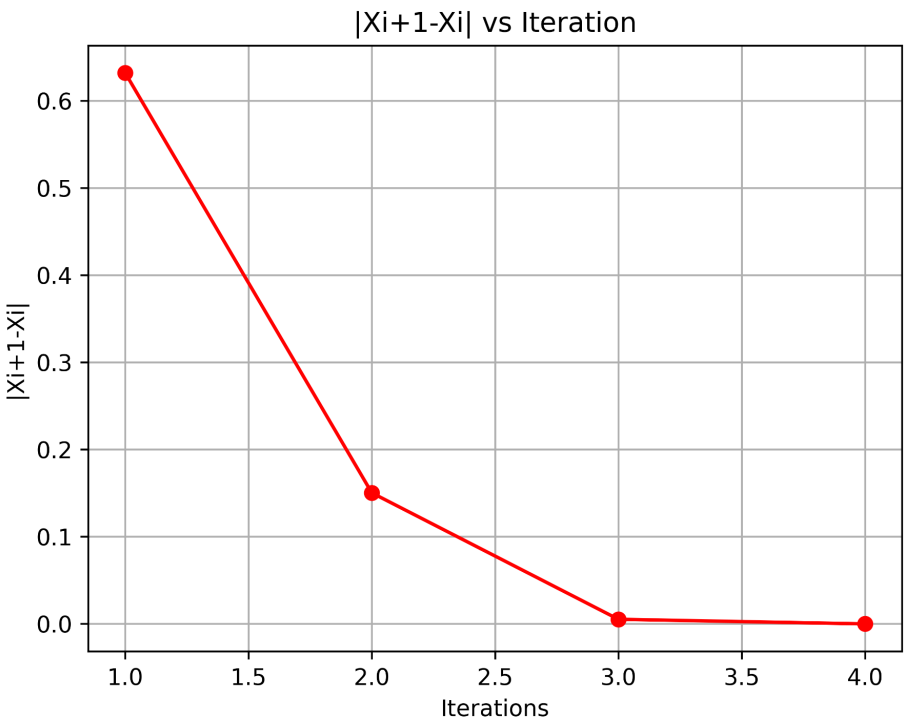
3. Observation-

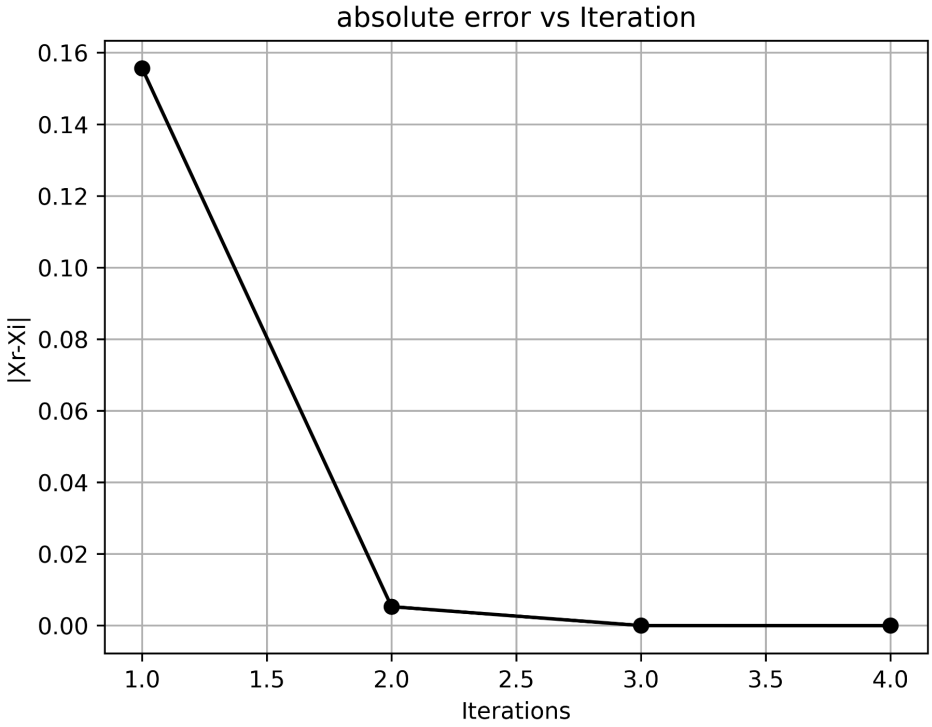
   1. After 5 iterations, the value of the approximated root is within the tolerance of the actual root = 0.5236
   2. The difference between successive roots keeps decreasing until it reaches the scale of machine epsilon, at which point the Python program can no longer distinguish the values.

4. Discussion-

   1. To prevent inexhaustive calculations (since the actual root may never be reached due to rounding error or machine epsilon) hence why a limit to iterations or tolerance value is set up.


Root vs Iteration
Root Value (Xr)
0.52
0.50
0.48
0.46
0.44
0.42
0.40
0.38
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Iterations


|Xi+1-Xi| vs Iteration
|Xi+1-Xi|
0.6
0.5
0.4
0.3
0.2
0.1
0.0
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Iterations


absolute error vs Iteration
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
|Xr-Xi|
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Iterations